

Lab Task 11

Shared Preferences — User Settings Manager

Android Studio • Kotlin • SharedPreferences API



Instructions

1. Complete the task following all requirements carefully.
2. After completion, push your project to your GitHub repository.
3. UI must be clean, aligned, and properly spaced.
4. SharedPreferences must persist data across app restarts — test by closing and reopening the app.



Scenario

You are building a User Settings Manager App for a university student portal. The app allows students to personalize their experience by saving preferences such as their name, theme, notification settings, language, and font size. These preferences are stored using SharedPreferences and must be retained even after the app is closed and reopened. The app demonstrates how to read, write, update, and clear preference data in Android using Kotlin.



Create a New Project

Project Name	UserSettingsApp
Template	Empty View Activity
Language	Kotlin
Package	com.university.usersettings

✓ Requirements & Steps

Screen 1 — Settings Dashboard (MainActivity)

Layout: ConstraintLayout + ScrollView

Required UI Elements:

- App header with university logo placeholder (ImageView) and title "Student Portal Settings"
- Student Name field — EditText to enter and save student name
- Theme selector — RadioGroup with options: Light, Dark, System Default
- Notifications toggle — SwitchCompat (enable/disable push notifications)
- Language selector — Spinner with options: English, Bangla, Arabic, French
- Font Size SeekBar — range 12 to 24, with a TextView showing current value (e.g., "Font Size: 16sp")
- Save Settings button — saves all values to SharedPreferences
- Reset to Default button — clears all saved preferences and resets UI
- View Saved Settings button — opens Screen 2 to display currently saved preferences

Screen 2 — Saved Settings Viewer (SettingsViewerActivity)

Layout: LinearLayout (vertical) + ScrollView

Display all saved preferences as labeled rows:

- Show each preference in a CardView row: label (bold) + saved value
- Display a "Last Saved:" timestamp using the saved System time
- If no settings saved yet, show a placeholder message: "No settings saved. Go back and save your preferences."
- Back button / Up navigation to return to MainActivity
- Edit button — navigates back to MainActivity to allow editing

Screen 3 — Profile Setup Screen (ProfileActivity)

Layout: ConstraintLayout

A dedicated screen to capture and persist student profile details:

- Student ID field — EditText (numbers only)
- Full Name field — EditText
- Department — Spinner: CSE, EEE, BBA, English, Law
- Year of Study — Spinner: 1st Year, 2nd Year, 3rd Year, 4th Year
- Email field — EditText with email input type
- Profile avatar — ImageView showing a default avatar icon (no camera required)
- Save Profile button — stores all values in a separate SharedPreferences file named "ProfilePrefs"
- A TextView banner at the top that greets the student: "Welcome back, [Name]!" (reads from saved profile on screen open)



SharedPreferences Requirements

Preference Files

- Use two separate SharedPreferences files:
 - **"AppSettings"** — stores theme, notifications, language, font size
 - **"ProfilePrefs"** — stores student name, ID, department, year, email
- Use getSharedPreferences() (not getPreferences()) to access named files
- Always use .apply() (async) rather than .commit() (synchronous) when saving

Keys to Store

Key Name	Data Type	Description
KEY_THEME	String	"light", "dark", or "system"
KEY_NOTIFICATIONS	Boolean	true/false toggle state
KEY_LANGUAGE	String	Selected language string
KEY_FONT_SIZE	Int	SeekBar value (12–24)
KEY_LAST_SAVED	Long	System.currentTimeMillis()
KEY_STUDENT_NAME	String	Full name from profile
KEY_STUDENT_ID	String	University student ID
KEY_DEPARTMENT	String	Selected department
KEY_YEAR	String	Year of study
KEY_EMAIL	String	Student email address



Code Hints

Reading & Writing Preferences

```
// Writing to SharedPreferences
val sharedPref = getSharedPreferences("AppSettings", Context.MODE_PRIVATE)
with(sharedPref.edit()) {
    putString("KEY_THEME", selectedTheme)
    putBoolean("KEY_NOTIFICATIONS", notifSwitch.isChecked)
    putInt("KEY_FONT_SIZE", seekBar.progress + 12)
    putLong("KEY_LAST_SAVED", System.currentTimeMillis())
    apply() // Always use apply(), not commit()
}

// Reading from SharedPreferences
val theme = sharedPref.getString("KEY_THEME", "light") // default: "light"
val notif = sharedPref.getBoolean("KEY_NOTIFICATIONS", true)
val fontSize = sharedPref.getInt("KEY_FONT_SIZE", 16)
```

Restoring UI from Saved Preferences (in onResume or onCreate)

```
override fun onResume() {
    super.onResume()
    val prefs = getSharedPreferences("AppSettings", Context.MODE_PRIVATE)
    // Restore theme radio button
    when (prefs.getString("KEY_THEME", "light")) {
        "light" -> rbLight.isChecked = true
        "dark" -> rbDark.isChecked = true
        "system" -> rbSystem.isChecked = true
    }
    // Restore SeekBar position
    seekBarFont.progress = prefs.getInt("KEY_FONT_SIZE", 16) - 12
    // Restore switch
    switchNotif.isChecked = prefs.getBoolean("KEY_NOTIFICATIONS", true)
}
```

Clearing All Preferences

```
// Reset to defaults
fun resetPreferences() {
    val prefs = getSharedPreferences("AppSettings", Context.MODE_PRIVATE)
    prefs.edit().clear().apply()
    // Also reset UI to default values
    rbLight.isChecked = true
    switchNotif.isChecked = true
    seekBarFont.progress = 4 // default 16sp
    spinnerLang.setSelection(0)
    Toast.makeText(this, "Settings reset to default",
        Toast.LENGTH_SHORT).show()
}
```



Navigation Requirements

- MainActivity → SettingsViewerActivity via "View Saved Settings" button (use Intent)
 - MainActivity → ProfileActivity via a floating action button or toolbar menu item
 - All activities must support Up/Back navigation (set `supportActionBar?.setDisplayHomeAsUpEnabled(true)`)
 - When returning to MainActivity from any screen, call `onResume()` logic to restore saved values into the UI
-